

Indian Institute of Technology Delhi

COL764: Information Retrieval and Web Search

Assignment 3: Rank Fusion and Learning to Rank

Submitted by

Aditya Anand
2023CS50284

Instructor

Prof. Srikanta Bedathur

October 24, 2026

Overview

The submission has four parts (Figure 1). Fusion and the two neural models are my own code; LambdaMART uses LightGBM's `lambdarank`. All numbers are nDCG@10.

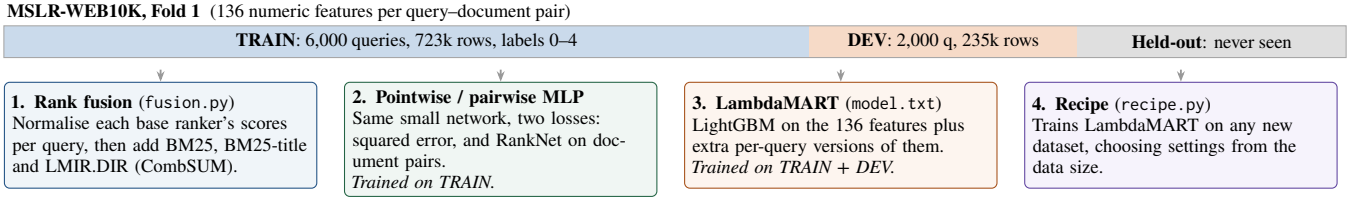


Figure 1: The four parts of the submission.

1 Rank fusion

Each candidate has four base-ranker scores: BM25, LMIR.DIR and LMIR.JM on the whole document, and BM25 on the title. Their scales differ (BM25 is positive, LMIR is a negative log-probability), so CombSUM and CombMNZ first **min–max normalise each channel inside the query** to $[0, 1]$ (written $\tilde{s}$). A channel where all documents score the same becomes all zeros. RRF uses 1-based ranks; ties keep the candidate order.

Method	TRAIN	DEV	DEV MAP@10
BM25 whole document alone	0.2835	0.2885	0.1205
RRF, $c = 60$	0.2926	0.2995	0.1246
RRF, $c = 150$ (best c)	0.2937	0.3002	0.1250
CombMNZ, 4 channels	0.2958	0.3061	0.1286
CombSUM, 4 channels	0.2961	0.3056	0.1285
CombSUM, 3 channels (shipped)	**0.3000**	**0.3105**	**0.1302**

Table 1: Fusion methods (nDCG@10 unless noted). DEV was only printed, never used to choose.

$$\mathrm{RRF}(d) = \sum_s \frac{1}{c + \mathrm{rank}_s(d)}, \qquad \mathrm{CombSUM}(d) = \sum_s \tilde{s}(d)$$

$$\mathrm{CombMNZ}(d) = \mathrm{CombSUM}(d) \cdot \left|\{s : \tilde{s}(d) > 0\}\right|$$

All choices were made on the 6,000 TRAIN queries: RRF with c from 1 to 500, CombSUM, CombMNZ, a z-score sum, and channel weights in $\{0, 0.25, 0.5, 1, 2\}$.

- **RRF is the weakest fusion.** It improves as c grows up to about 150 (0.2843 at $c = 1$), then stays flat. It keeps only the order of each list and throws away how far apart the scores are.
- **CombMNZ ≈ CombSUM.** After min–max, only the single lowest document of each channel gets 0, so the MNZ multiplier is almost always 4 and changes nothing.
- **LMIR.JM is dropped.** The weight search gave it weight 0. It is the weakest single channel (0.2621) and mostly repeats LMIR.DIR. To check that picking from about 400 options did not overfit, I chose on 4 TRAIN folds and scored the 5th: 0.2989 ± 0.0041, close to 0.3000.

2 Pointwise, pairwise and LambdaMART

	Pointwise	**Pairwise (RankNet)**	**LambdaMART**
One example	one document	two documents of the *same* query, different labels	a whole query
Loss	$(s - y)^2$, label $y \in \{0, \ldots, 4\}$	$\log(1 + e^{-(s_i - s_j)})$, i more relevant; weighted by $\lvert\Delta\mathrm{nDCG}_{ij}\rvert$ in training	lambdas: RankNet gradient $\times \lvert\Delta\mathrm{nDCG}_{ij}\rvert$
Model	MLP 136→64→1	MLP 136→256→256→256→1 (largest Track P allows)	2,835 trees, 127 leaves

Pairwise training. Pairs come only from one query, ties are dropped, and the better document is first. Each batch is 64 whole queries with up to 4,000 pairs per query (picked more often when the gain gap $2^{y_i} - 2^{y_j}$ is large). Each pair is weighted by $|\Delta\mathrm{nDCG@10}|$, the change in nDCG if the two documents swapped places. The graded `ranknet_loss` stays plain and unweighted, and uses `logaddexp` so a huge score gap cannot overflow. Features are clipped to their 1st–99th percentile and standardised (fit on training rows only). Both networks use Adam, early-stop on a fixed tenth of TRAIN and keep the best epoch; DEV is scored once, at the end.

What a lambda is. For a pair where i is more relevant, RankNet's gradient says "push s_i up, push s_j down". LambdaMART multiplies that push by $|\Delta\mathrm{nDCG}_{ij}|$: a swap at ranks 1 and 2 matters a lot, a swap at ranks 80 and 90 almost not at all. The sum of all pushes on a document is its *lambda*. Each new regression tree is fit to the lambdas and added with a small learning rate; then scores and lambdas are recomputed.

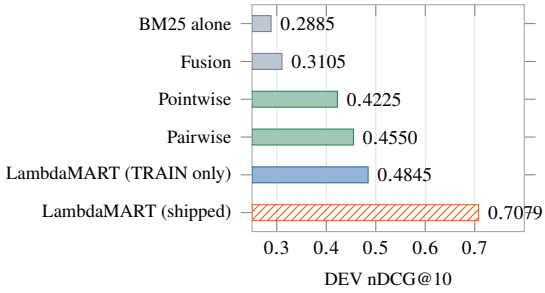


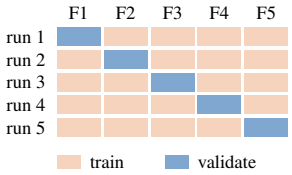
Figure 2: DEV nDCG@10. The hatched bar is *not* a fair score: DEV was inside its training data.

Comparison (Figure 2). Pairwise beats pointwise by 0.033. Pointwise looks at each document alone, so an error at rank 1 costs the same as one at rank 50. In an early run with the *same* 64-unit network, plain RankNet and pointwise were almost equal on DEV (0.4266 vs. 0.4297), so the gain came from the $|\Delta\mathrm{nDCG}|$ weights and the wider network. **LambdaMART is best:** fit on TRAIN only with the CV-chosen setting (Section 3), it scores 0.4845 on DEV. The shipped file's 0.7079 is an in-training

score (DEV was in its fit), so it is not compared.

3 Cross-validation and the generalisation gap

I do not have the held-out queries, so I measure the gap with **grouped 5-fold cross-validation inside TRAIN** (right). All documents of a query go to the same fold, so no query is on both sides. Each run trains on 4,800 queries and validates on 1,200. One model per fold is grown to the largest tree count and scored at fixed counts 100, 200, 300, 500, 750, 1000, 1500, 2000 (`predict(num_iteration=n)`), so **nothing is early-stopped on the fold I report**. Trees need no feature scaling, so nothing fitted leaks across folds. I report the mean ± std over the five validation folds and the val/train ratio.



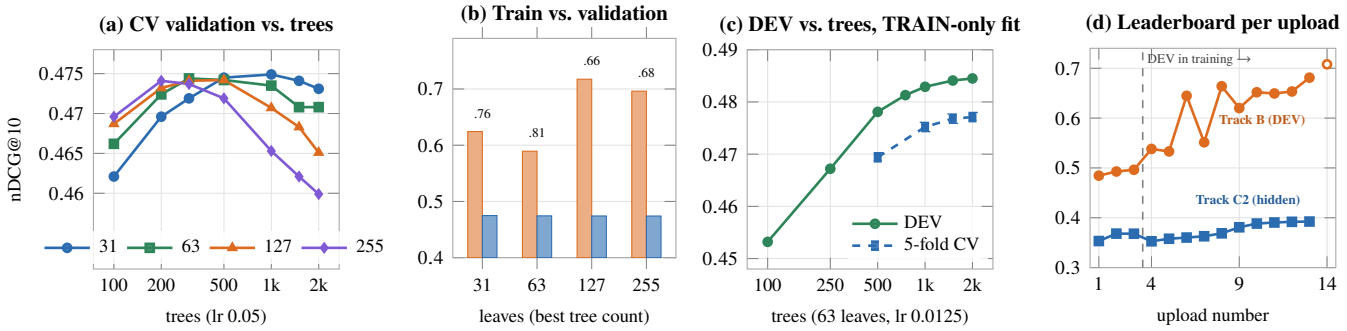


Figure 3: In (b) and (d), orange = data the model was fit on, blue = data it never saw. (a)–(b) Grouped 5-fold CV on TRAIN; curves in (a) are leaf counts, numbers on (b) are val/train ratios. (c) A LambdaMART fit on TRAIN only, scored on DEV after the choice was made, next to its CV estimate. (d) Real leaderboard scores of my LambdaMART uploads in time order (14 = shipped file; its C2 score is not shown).

lr / leaves / min leaf / cols	trees	val nDCG@10	train	ratio
0.05 / 31 / 100 / 0.8	1000	0.4749 ± .0059	0.624	0.76
0.05 / 63 / 100 / 0.8	300	0.4744 ± .0063	0.589	**0.81**
0.05 / 127 / 100 / 0.8	500	0.4742 ± .0077	0.717	0.66
0.05 / 255 / 100 / 0.8	200	0.4741 ± .0059	0.696	0.68
0.05 / 255 / 100 / 0.8	2000	0.4599	0.944	0.49
0.05 / 63 / 500 / 0.5	500	0.4741 ± .0068	0.623	0.76
0.025 / 63 / 500 / 0.5	1000	0.4766 ± .0070	0.625	0.76
0.0125 / 63 / 500 / 0.5	**2000**	**0.4771 ± .0068**	0.626	0.76

Table 2: Grouped 5-fold CV on TRAIN, at each setting's best tree count (± = std over folds). The orange row is the same 255-leaf model left to 2,000 trees.

What the gap shows.

- **Validation is flat; training is not.** Every tree shape peaks at 0.474–0.477 (Fig. 3a). Bigger trees only raise the training score: 255 leaves reach 0.944 on the training folds while validation *falls* to 0.460. Extra capacity memorises queries.
- **Big trees need few rounds:** 255 leaves peak at 200 trees, 31 leaves at 1,000.
- **A lower learning rate helps a little:** 0.05→0.025 improved all 5 folds (+0.0025), 0.0125 added +0.0005. Both are below the fold std (≈0.007).
- **DEV agrees with CV for a model that never saw DEV:** 0.4845 vs. 0.4771, with the same curve shape (Fig. 3c). The first pairwise model too: 0.4364 on DEV, 0.4311 on the held-out split (Track P).
- **Once DEV is in training, its score means little.** Over my uploads (Fig. 3d) Track B rose by 0.20, the hidden C2 test only from 0.353 to 0.392. My estimate for new clean queries stays ≈**0.477**.

4 Regularisation choices and the shipped LambdaMART

Clean CV says more capacity does not help unseen queries, so I spent it on **robustness**: Track C2 runs the same file on five changed copies of the held-out data. A clue to those changes: Gaussian noise of 0.05× each feature's std drops the plain TRAIN-only model on DEV from 0.4845 to 0.3450, close to its real C2 score of 0.3533. My responses:

- **Usual tree regularisation:** learning rate falling 0.05→0.015, column sample 0.5, row bagging 0.9, L2 0.1, at least 20–40 rows per leaf, tree count fixed in advance.
- **Scale-free columns.** Each feature also gets its *percentile inside the query* (unchanged by any increasing transform, e.g. log or ×10) and its *z-score inside the query* (unchanged by shift or positive scaling): 408 columns, rebuilt by `rerank_with_ltr` at scoring time.
- **Corrupted training copies:** Gaussian or multiplicative noise, $\log(1+|x|)$, cells set to 0 or missing, the five fields of a feature group shuffled, one or two fields zeroed, per-query rescaling. Labels stay the same; each copy gets new query ids.

Probe (unseen TRAIN queries)	clean	corrupted
raw + percentile columns	0.4727	0.3620
+ within-query z-score	**0.4836**	**0.4187**
no corrupted copies	**0.5653**	0.3465
+ corrupted copies	0.5222	**0.4558**

Table 3: Two small probes (different sizes; compare within a block). Corrupted = mean over 7–8 kinds of corruption. DEV not read.

Table 3 shows the trade-off. The z-score columns help on both clean and corrupted data. The corrupted copies **cost 0.043 on clean data but gain 0.109 under corruption**. A column sample of 0.5 instead of 0.8 also lost less when 10% of features were zeroed (−0.022 vs. −0.024), at the same clean score. Since clean CV showed leaf count barely matters, it was set by these probes, where larger trees did better under corruption; the shipped file uses 127 leaves. During the staged run below, the real C2 score rose from 0.381 to 0.392 (uploads 9–13, Fig. 3d).

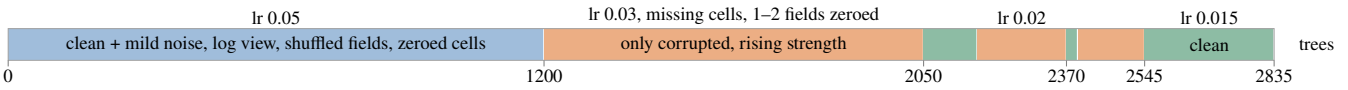
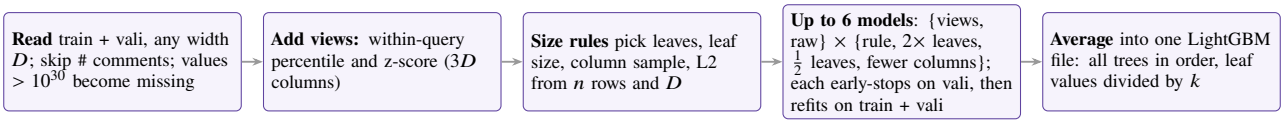


Figure 4: How the 2,835 shipped trees were grown: 11 stages, each continuing from the last file, all on TRAIN + DEV. Orange stages use only corrupted copies; green stages use only clean rows; the last 250 trees are clean.

5 Training recipe for the hidden datasets (Track C1)



An MSLR model cannot be used on the hidden datasets (different, unnamed features, up to ~300), so the recipe must carry *how* to train. With n training rows and d input columns: leaves = clip($2^{\text{round}(\log_2(\sqrt{n}/16))} - 1$, 7, 63); min leaf = clip($n/4000$, 20, 200); column sample = clip($1 - 0.15\log_2(d/32)$, 0.5, 1); L2 = 10 if $n < 50{,}000$, else 1; learning rate 0.05. Small data gets small, strongly regularised trees, and nothing hard-codes 136 features. A cost model based only on data size decides how many models fit in a 15-minute budget (staff limit: 20), so the output depends only on the data and the seed. On 5 benchmark-like tasks cut from TRAIN, an earlier single-model version of these rules averaged 0.5011 vs. 0.4882 for the starter recipe, and the refit helped on all 5. A full run on MSLR TRAIN + DEV (958k rows) took 313 s and 4.0 GB; a synthetic file at the stated limit (500k rows, 300 features) took 353 s and 5.2 GB.

6 Final configuration and what I traded off

Fusion is CombSUM of three min–max channels. The **pointwise** and **pairwise** networks use only TRAIN. The **LambdaMART** file has 2,835 trees with 127 leaves on 408 columns, fit on TRAIN + DEV with corrupted copies. For expected generalisation I gave up two things. First, *a fair DEV score*: training on DEV lifts Track B to 0.7079, but that is in-training; my estimate for unseen clean queries is the CV value of about 0.477 (0.4845 on DEV for a TRAIN-only fit). Second, *some clean accuracy*: in the probe the corrupted copies cost about 0.04 on clean data and gain about 0.11 under corruption, which is what Track C2 measures. The 63-leaf clean model has the best val/train ratio (0.81) at the same 0.477, but no defence against noisy, missing or rescaled features, so I did not ship it. For Track C1 I used rules and an average of several tree shapes, so no single MSLR-tuned constant decides the result on data I cannot see.

Correctness of the hand-written parts. The three fusion functions match the worked examples in their docstrings. `ranknet_loss([2,0],[0,0])` gives 0.410038, and a score gap of −2000 gives a loss of 2000 instead of overflowing. `make_pairs` returns only same-query, strictly ordered pairs, better document first, capped per query. One `pairwise_step` raises $s_i - s_j$, and two runs with the same seed give identical weights. The training splits of every submitted model are:

Model	Trained on	Tuned on	Early stopping
LambdaMART `model.txt`	TRAIN + DEV	5-fold CV on TRAIN; tree count and copies on TRAIN slices and DEV	none
`pointwise_model.pt`	9/10 of TRAIN	not tuned	other 1/10 of TRAIN
`pairwise_model.pt`	9/10 of TRAIN	pair sampling and learning rate on a TRAIN slice	other 1/10 of TRAIN
`recipe.py`	hidden dataset's train + vali	MSLR TRAIN CV and slices; synthetic transfer sets	its vali, then refit

7 AI use and code provenance

AI use. I used Claude Code (Anthropic, Claude Opus 5.5) and Cursor (Grok) to read the starter code, write the submission, run the experiments and draft this report. I chose the fusion channels, the LambdaMART settings and the recipe rules, and I checked the numbers. I am responsible for the submission. **Code provenance.** No external code was copied. RRF follows Cormack, Clarke and Buettcher (SIGIR 2009); CombSUM and CombMNZ follow Fox and Shaw (TREC-2, 1994); RankNet follows Burges et al. (ICML 2005); the |ΔnDCG| weights follow Burges (MSR-TR-2010-82) and are used only outside the graded `ranknet_loss`. LambdaMART is LightGBM's `lambdarank`. Libraries: NumPy, LightGBM, PyTorch.